

Job Alert Platform - System Architecture & Workflow

Acadeno Technologies - Automated Job Alert System

1. Working of the Project (In Simple Terms)

The Job Alert Platform is an automated email notification system built to connect students with verified IT career opportunities. It automates the collection of student subscriptions, cloud secret

Real-World Analogy: How the System Works

1. Admin Portal: System admins register student subscribers through a Flask web application.
2. Cloud Secret Vault: Student lists are encrypted with PyNaCl and pushed to GitHub Secrets.
3. Automated Mailer Engine: Reads jobs from jobs.json, picks 20 random openings + career quote, formats responsive HTML cards, and emails each student via Gmail SMTP.

2. Key Models Used in the System

A. User Authentication Data Model (SQLite: users table)

Stores admin credentials: id, username, password (Bcrypt hash), role (admin/user), email.
Enforces Role-Based Access Control so only authorized admins can add/delete students.

B. Student Subscriber Data Model (SQLite: students table)

Stores recipient directory: id, name, email (UNIQUE constraint), created_at timestamp.
Calculates enrollment duration in days. Uses UPSERT logic on duplicate email submission.

C. Job Opportunity Data Model (jobs.json)

Datastore of scraped IT job listings with title (e.g., AI Engineer, Data Science Intern) and link.
Links are sanitized in mailer.py to guarantee valid HTTPS application URLs.

D. Career Quote Data Model (career_quotes_unique.xlsx)

Indexed Excel dataset of motivational career quotes parsed by pandas for daily email rotation.

E. Sodium SealedBox Cryptographic Security Model (PyNaCl)

Asymmetric Curve25519 + XSalsa20 + Poly1305 public-key encryption model. When students are added/deleted, `sync_student_secrets()` encrypts name and email payloads using GitHub's public key before updating GitHub Repository Secrets via REST API.

F. Bcrypt Hashing & Timed Serializer Security Models

1. Bcrypt Hashing: Salted blowfish cipher hashing for user passwords.
2. URLSafeTimedSerializer: Signed HMAC SHA-1 tokens with 1-hour expiration for password reset.

3. Step-by-Step Output Pipeline (Data Flow)

Step 1: Student Registration & Cloud Encryption Sync

Admin logs into Flask portal -> Submits student details -> Saved to SQLite `users.db` -> `sync_student_secrets()` encrypts list with GitHub Public Key -> Pushes to GitHub Secrets API.

Step 2: Recipient Data & Content Ingestion

`mailer.py` executes -> Reads `EMAIL_TO` and `STUDENT_NAMES` from environment secrets (or local DB) -> Loads `jobs.json` repository -> Parses `career_quotes_unique.xlsx` via pandas.

Step 3: Dynamic Job Sampling & URL Sanitization

`mailer.py` calls `random.sample(jobs, min(20, len(jobs)))` to select 20 fresh job openings -> Fixes malformed URLs to ensure clean application links.

Step 4: Responsive HTML Email Newsletter Rendering

Generates styled HTML card layout with purple 'View & Apply' buttons -> Replaces `{USER_NAME}` placeholder dynamically for each student recipient.

Step 5: Secure SMTP Dispatch & Output Delivery

Connects to Gmail SMTP (`smtp.gmail.com:587`) using STARTTLS -> Authenticates with credentials -> Sends individual MIME HTML emails straight to each student's Gmail inbox.

4. Summary Table of Core Project Files

File Name	Primary Role	Tech Stack
auth_app.py	Flask Web Portal, Auth, Secret Sync	Flask, PyNaCl, SQLite
create_db.py	Database Schema & Admin Setup	SQLite3, Flask-Bcrypt
mailer.py	Job Sampler, Email Builder & Dispatcher	smtplib, Pandas, MIME
jobs.json	Job Openings Repository	JSON Datastore
render.yaml	Cloud Deployment Blueprint	Gunicorn / Render

5. System Architecture & Data Flow Diagram

